

BUILD-OVERVIEW.md

Version : V1

Statut : Document racine du Build Blueprint

Type : Guide d'orchestration du développement

1. Objectif

Ce document définit l'ordre officiel de construction de la plateforme Vitala.

Il constitue le point d'entrée de tout développement réalisé par une équipe humaine ou une Intelligence Artificielle.

Aucun développement ne doit commencer sans respecter les dépendances décrites dans ce document.

2. Principes fondamentaux

Le développement de Vitala repose sur les principes suivants :

- Domain-Driven Design (DDD)
- Architecture modulaire
- Offline First
- API First
- Security by Design
- AI-Ready Development
- Documentation First

Chaque module doit être terminé et validé avant de passer au suivant.

3. Ordre officiel de construction

Le développement doit respecter les phases suivantes.

Phase 1 — Fondation

Objectif : mettre en place les bases techniques.

- Initialisation des dépôts
- Configuration Flutter
- Configuration Backend
- Configuration Supabase
- Gestion des environnements

- CI/CD
- Structure des dossiers
- Outils qualité (lint, format, tests)

Livrable :

Une architecture vide mais entièrement opérationnelle.

Phase 2 — Infrastructure

Construire les services transverses.

- Authentification
- Unified Digital Identity (UDI)
- Base de données
- Storage
- Media
- Notifications
- Journalisation (Audit)
- Configuration
- Gestion des erreurs

Livrable :

Toutes les fondations techniques sont disponibles.

Phase 3 — Domaines métier

Construire les domaines dans l'ordre suivant :

1. UDI
2. Flash
3. Mission
4. Trust

Chaque domaine comprend :

- Modèles
- Services
- Repositories
- API
- Validation
- Tests

Aucun domaine ne peut dépendre d'un domaine non terminé.

Phase 4 — Synchronisation Offline

Construire :

- Local Workspace
- Drafts
- Outbox
- Sync Engine
- Conflict Resolution
- Checkpoints

Cette phase est obligatoire avant les moteurs d'intelligence.

Phase 5 — Moteurs d'intelligence

Construire :

- Radar
- Veille
- Recommendation
- Search

Ces moteurs ne doivent jamais modifier directement les domaines métier.

Ils produisent uniquement des analyses, projections ou recommandations.

Phase 6 — Interface utilisateur

Construire progressivement :

- Design System
- Composants
- Écrans
- Navigation
- États
- Connexion aux APIs

L'interface doit consommer exclusivement les APIs officielles.

Phase 7 — Intégration

Valider :

- Frontend ↔ Backend
- Backend ↔ Base de données
- Offline ↔ Sync

- Intelligence ↔ Domaines
-

Phase 8 — Qualité

Exécuter :

- Tests unitaires
 - Tests d'intégration
 - Tests Offline
 - Tests de performance
 - Tests de sécurité
 - Revue d'architecture
-

Phase 9 — Préparation Production

Configurer :

- Monitoring
 - Sauvegardes
 - Observabilité
 - Déploiement
 - Documentation finale
-

4. Dépendances officielles

Les dépendances entre modules sont les suivantes :

- Infrastructure avant Domaines.
- Domaines avant Synchronisation.
- Synchronisation avant Intelligence.
- Intelligence avant Interface complète.
- Interface avant Validation finale.

Aucun module ne doit contourner cette chaîne.

5. Critères de validation d'une phase

Une phase est considérée comme terminée uniquement si :

- toutes les fonctionnalités prévues sont développées ;
- les tests passent avec succès ;
- la documentation est mise à jour ;
- aucune dette technique critique n'est ouverte ;

- les règles définies dans "AI Development Rules" sont respectées.
-

6. Documents obligatoires à consulter

Avant de développer un module, les documents suivants doivent être lus :

- Vision Project Specification (VPS)
 - Domain Model
 - API Contracts
 - Database
 - Development Guide
 - AI Development Rules
 - Le Build Blueprint correspondant au module
-

7. Règles absolues

Il est interdit :

- de développer directement depuis une idée non documentée ;
 - de modifier un domaine sans mettre à jour sa documentation ;
 - de créer des dépendances circulaires entre domaines ;
 - de contourner les APIs officielles ;
 - de mélanger logique métier et interface utilisateur.
-

8. Définition de réussite

Le Build Blueprint est considéré comme correctement appliqué lorsque :

- chaque domaine est autonome ;
- chaque API respecte son contrat ;
- chaque table appartient à un seul domaine ;
- l'architecture reste modulaire ;
- une IA ou un développeur peut reconstruire Vitala sans ambiguïté à partir de la documentation officielle.